

Quick Start - Simulation Mode

Get started with Franka User Control in simulation mode (no robot required!).

Prerequisites

- Ubuntu 22.04 or compatible
- ROS2 Humble installed
- This package built successfully

Step 1: Build the Package

```
bash

cd ~/franka_user_control_ws
colcon build --packages-select franka_user_control
source install/setup.bash
```

Step 2: Launch Simulation

```
bash

ros2 launch franka_user_control simulation.launch.py
```

You should see:

```
[simulation_bridge_node]: Simulation bridge node started
[simulation_bridge_node]: Ready to accept commands
[fake_robot_state_publisher]: Fake robot state publisher started
```

Step 3: Run a Test

Open a **new terminal**:

```
bash

source ~/franka_user_control_ws/install/setup.bash
ros2 run franka_user_control test_simulation.py
```

The robot will move in simulation! You'll see output like:

```
Sending velocity: [0.050, 0.000, 0.000] for 2.0s
Velocity command completed
```

Step 4: Try the Python API

Create a test file `my_test.py`:

```
python

#!/usr/bin/env python3
import time
import rclpy
from geometry_msgs.msg import Twist
from rclpy.node import Node

# Initialize ROS2
rclpy.init()
node = Node('my_test')

# Create velocity publisher
pub = node.create_publisher(Twist, 'velocity_command', 10)
time.sleep(0.5) # Let publisher connect

# Send velocity command
cmd = Twist()
cmd.linear.x = 0.05 # 5 cm/s forward
print("Moving forward...")

for _ in range(200): # 2 seconds at 100 Hz
    pub.publish(cmd)
    time.sleep(0.01)

# Stop
cmd.linear.x = 0.0
pub.publish(cmd)
print("Stopped!")

# Cleanup
rclpy.shutdown()
```

Run it:

```
bash

python3 my_test.py
```

What's Happening?

1. **Fake Robot State Publisher** simulates joint states
2. **Simulation Bridge Node** accepts commands and simulates motion
3. **Your commands** move the simulated robot
4. **Joint states** are published just like the real robot

Next Steps

Try More Tests

```
bash

# Velocity control patterns
ros2 run franka_user_control test_simulation.py

# Monitor robot state
ros2 topic echo /joint_states

# View current pose
ros2 topic echo /current_pose
```

Visualize with RViz (Optional)

```
bash

# Launch with RViz
ros2 launch franka_user_control simulation.launch.py use_rviz:=true
```

Use the Full Python API

Once comfortable with simulation, the same API works on real hardware:

```
python
```

```
from franka_python_interface import FrankaInterface
```

```
robot = FrankaInterface()  
robot.connect()
```

```
# All these work in simulation AND on real robot:  
robot.set_cartesian_velocity(vx=0.05, duration=2.0)  
robot.move_relative(dx=0.1, dy=0.0, dz=0.0)  
robot.stop()  
  
robot.disconnect()
```

Troubleshooting

"Package not found"

```
bash  
  
# Make sure you sourced the workspace  
source ~/franka_user_control_ws/install/setup.bash
```

"No executable found"

```
bash  
  
# Make sure scripts are executable  
chmod +x ~/franka_user_control_ws/src/franka_user_control/scripts/*.py  
  
# Rebuild  
cd ~/franka_user_control_ws  
colcon build --packages-select franka_user_control
```

"No topics published"

```
bash  
  
# Check if nodes are running  
ros2 node list  
  
# Should see:  
# /fake_robot_state_publisher  
# /simulation_bridge_node
```

What's Next?

- 1. **Read the full documentation:** See `README.md`
- 2. **Try more test scripts:** `test_velocity.py`, `test_position.py`, `test_combined.py`
- 3. **Learn about safety:** See `docs/simulation_guide.md`
- 4. **When ready for real robot:** See `README.md` hardware section

Key Differences: Simulation vs Real Robot

Aspect	Simulation	Real Robot
Launch command	<code>simulation.launch.py</code>	<code>user_control.launch.py robot_ip:=X.X.X.X</code>
Safety reflexes	Simulated	Hardware-enforced
Force feedback	Not available	Available
Control frequency	~100 Hz	1000 Hz
Risk level	Zero	Requires caution

Tips

- **Start with simulation** to learn the API safely
- **Test all trajectories** in simulation first
- **Use conservative limits** when moving to real hardware
- **Keep emergency stop** accessible on real robot

Happy simulating! 🤖